



Gobierno Bolivariano
de Venezuela

Ministerio del Poder Popular
para la Educación Universitaria

Universidad Nacional Experimental
para las Telecomunicaciones e Informática (UNETI)



República Bolivariana De Venezuela

Universidad Nacional Experimental Para Las Telecomunicaciones E Informática

Vicerrectorado Académico

Programa Nacional De Formación En Ingeniería En Informática

Desarrollo de la Aplicación Móvil "AdoptaPet"

Alumna:

Yhusleika Molina CI: 27401354

Caracas, Mayo de 2026



Indice

Introducción	3
Marco Tecnológico	4
Arquitectura del Sistema.....	4
Estructura de Archivos y Modularización.....	4
Navegación y Sistema de Rutas	6
Implementación de Páginas	6
Inicio.....	6
Perfil del Adoptante (Información Personal)	7
Contacto	9
Diseño Visual y Personalización	10
Guía de Ejecución	11
Tecnologías Utilizadas.....	12
Estructura del Proyecto	12
Servicios y Lógica Compartida.....	13
Flujo de Datos	13
Conclusión	15
Bibliografía	16



Introducción

El desarrollo de "AdoptaPet" nace como una respuesta tecnológica a la necesidad de conectar refugios de animales con personas interesadas en la adopción responsable. A diferencia de una aplicación estática, este proyecto busca demostrar cómo las tecnologías híbridas modernas (Ionic + Angular + Capacitor) permiten construir experiencias móviles nativas utilizando conocimientos de desarrollo web. Este documento explica en detalle el funcionamiento interno de la aplicación, desde la estructura de archivos hasta las decisiones de código implementadas manualmente, con énfasis en la lógica de programación que va más allá de la configuración automática generada por los frameworks.



Marco Tecnológico

Para la materialización de AdoptaPet se empleó un conjunto de herramientas que facilitan el desarrollo multiplataforma:

- **Ionic Framework v7+:** Biblioteca de componentes UI optimizados para móviles, que provee elementos como ion-menu, ion-card, ion-segment, ion-toast, entre otros. Cada componente se adapta automáticamente a los estilos de iOS (Apple) o Android (Material Design) según la plataforma.
- **Angular v17+:** Framework SPA (Single Page Application) que actúa como motor lógico y estructural. Maneja el enrutamiento, la inyección de dependencias, los módulos, los componentes y los formularios reactivos. Se utiliza la sintaxis moderna de control flow (@for, @if) y el sistema de NgModules para la carga diferida.
- **TypeScript:** Superset tipado de JavaScript que garantiza la integridad del código mediante tipado fuerte, permitiendo detectar errores en tiempo de compilación y mejorando la mantenibilidad.
- **Capacitor:** Capa nativa que actúa como puente entre la aplicación web y el hardware del dispositivo. Permite acceder a la cámara, al sistema de archivos y generar archivos APK (Android) o IPA (iOS) sin reescribir el código fuente.

Arquitectura del Sistema

La arquitectura de AdoptaPet se fundamenta en un modelo basado en componentes jerárquicos y servicios compartidos. Esta separación de responsabilidades asegura que la interfaz gráfica permanezca independiente de la lógica de negocio y del acceso a los recursos del dispositivo, facilitando la escalabilidad futura de la aplicación.

Estructura de Archivos y Modularización



El proyecto se organiza de manera modular dentro del directorio src/app, centralizando la lógica de negocio. Cada pantalla principal (Inicio, Perfil, Contacto) posee su propio módulo con carga diferida (lazy loading), lo que minimiza el tiempo de carga inicial ya que el navegador o dispositivo solo descarga el código necesario para la ruta que se está visualizando.

La estructura relevante del proyecto es la siguiente:

AdoptaPet/	
— src/	
— app/	
— app.component.html	(Plantilla del menú lateral)
— app.component.ts	(Lógica principal)
— app.module.ts	(Módulo raíz)
— app-routing.module.ts	(Rutas principales con lazy loading)
— services/	
— theme.service.ts	(Servicio de modo oscuro)
— inicio/	(Módulo de la página de inicio)
— informacion-personal/	(Módulo del perfil / formulario)
— contacto/	(Módulo de contacto)
— assets/	(Imágenes y recursos estáticos)
— theme/	
— variables.scss	(Paleta de colores y variables CSS)
— global.scss	(Estilos globales y keyframes)
— index.html	(Punto de entrada HTML)
— main.ts	(Bootstrap de Angular)
— capacitor.config.ts	(Configuración nativa de Capacitor)
— ionic.config.json	(Configuración del CLI de Ionic)
— package.json	(Dependencias de NPM)

Módulos y carga diferida:

Cada ruta definida en app-routing.module.ts utiliza loadChildren en lugar de component, de modo que Angular compile cada página en un chunk de JavaScript separado. El chunk solo se descarga cuando el usuario navega a esa ruta por primera



vez. Esto reduce drásticamente el tiempo de carga inicial, descargando únicamente el código de la página de inicio.

Navegación y Sistema de Rutas

La interfaz principal se construye sobre un menú lateral retráctil (ion-menu) combinado con un ion-router-outlet. El comportamiento es adaptativo: en pantallas grandes ($\geq 992\text{px}$) el menú permanece visible junto al contenido; en móviles se oculta automáticamente para priorizar el espacio de lectura.

Rutas definidas y lazy loading:

" → redirige a 'inicio'

'inicio' → InicioPageModule

'informacion-personal' → InformacionPersonalPageModule

'contacto' → ContactoPageModule

El componente raíz (AppComponent) actúa como el cascarón principal que envuelve toda la experiencia. En su plantilla se declara la estructura del menú lateral y el contenedor de salida donde Angular renderiza dinámicamente las demás pantallas.

Implementación de Páginas

A continuación, se explica el funcionamiento interno de cada página, destacando las decisiones de código implementadas manualmente.

Inicio

La pantalla de bienvenida no solo es informativa, sino que integra una gráfica de barras animada desarrollada completamente con CSS nativo. Para evitar la sobrecarga de bibliotecas externas (como Chart.js), se implementó el siguiente código en inicio.page.scss:

```
.bar {  
  width: 20px;  
  background: var(--ion-color-primary);  
  border-radius: 4px 4px 0 0;  
  animation: barGrow 1s ease-out forwards;  
  transform-origin: bottom;  
}  
  
.bar-container span {  
  margin-top: 8px;  
  font-size: 12px;  
  color: var(--ion-color-medium);  
}  
  
@keyframes barGrow {  
  from { transform: scaleY(0); }  
  to { transform: scaleY(1); }  
}  
  
@keyframes float {  
  0% { transform: translateY(0px); }  
  50% { transform: translateY(-10px); }  
  100% { transform: translateY(0px); }  
}
```

Interpretación:

Mediante la propiedad `transform: scaleY()` y estableciendo el punto de origen en `bottom`, se logra que el navegador utilice la aceleración por hardware (GPU) para animar el crecimiento de las barras. Esto garantiza 60 FPS fluidos incluso en dispositivos móviles de gama baja, demostrando un óptimo control del repintado del DOM.

Además, la página incluye accesos rápidos (adopciones, refugios, donaciones) y un listado dinámico de historias de éxito renderizado con la directiva `@for`.

Perfil del Adoptante (Información Personal)

Esta sección actúa como el formulario principal de adopción. Dado que el formulario es extenso, se implementó un sistema de pestañas lógicas usando `ion-segment` y `ngSwitch`.

Fragmento en `informacion-personal.page.html`:

```
<div class="segment-container">
  <ion-segment [(ngModel)]="currentTab" mode="md">
    <ion-segment-button value="personal">
      <ion-label>Personal</ion-label>
    </ion-segment-button>
    <ion-segment-button value="hogar">
      <ion-label>Hogar</ion-label>
    </ion-segment-button>
    <ion-segment-button value="preferencias">
      <ion-label>Preferencias</ion-label>
    </ion-segment-button>
  </ion-segment>
</div>

<div [ngSwitch]="currentTab">
  <!-- Pestaña Personal -->
  <ion-card class="premium-card" *ngSwitchCase="'personal'">...
</ion-card>

  <!-- Pestaña Hogar -->
  <ion-card class="premium-card" *ngSwitchCase="'hogar'">...
</ion-card>

  <!-- Pestaña Preferencias -->
  <ion-card class="premium-card" *ngSwitchCase="'preferencias'">...
</ion-card>
</div>
```

Interpretación:

Se enlaza la variable `currentTab` mediante doble enlace de datos (`ngModel`). La directiva estructural `*ngSwitchCase` evalúa condicionalmente esta variable: Angular solo inserta en el DOM los elementos de la pestaña activa, destruyendo los demás. Esto representa una gestión de memoria superior frente al uso de clases CSS de ocultamiento (`display: none`).

Integración con la cámara nativa:

La página permite al usuario seleccionar una foto de perfil desde la galería del dispositivo mediante la API de Capacitor.


```
import { Camera, CameraResultType, CameraSource } from '@capacitor/camera';

> @Component({ ...
})
export class InformacionPersonalPage implements OnInit {
  currentTab = 'personal';
  profileImage = 'https://ionicframework.com/docs/img/demos/avatar.svg';

  constructor(public theme: ThemeService) { }

  ngOnInit() {
  }

  async changeProfilePicture() {
    try {
      const image = await Camera.getPhoto({
        quality: 90,
        allowEditing: true,
        resultType: CameraResultType.Uri,
        source: CameraSource.Photos // Selección desde la galería
      });

      if (image.webPath) {
        this.profileImage = image.webPath;
      }
    } catch (error) {
      console.log('Selección de foto cancelada o con error', error);
    }
  }
}
```

Interpretación:

Se programó una función asíncrona (async/await) para prevenir el bloqueo de la interfaz (hilo principal) mientras el sistema operativo procesa la solicitud. Es crucial la configuración source: CameraSource.Photos, ya que instruye directamente al sistema operativo a abrir el explorador de archivos y la galería en lugar de activar la cámara, mejorando la usabilidad cuando el usuario desea buscar una foto preexistente.

Contacto

La página de contacto está diseñada para conectar al usuario con los refugios mediante enlaces profundos (deep links) que abren aplicaciones de terceros.

Fragmento en contacto.page.html:



```
<ion-list lines="none" class="ion-margin-top support-list">
  <ion-list-header>Otras formas de contacto</ion-list-header>
  <ion-item detail="true" button href="https://wa.me/1234567890?text=Hola,%20me%20gustar%C3%AD%20adoptar" target="_blank">
    <ion-icon name="logo-whatsapp" slot="start" color="success"></ion-icon>
    <ion-label>WhatsApp</ion-label>
  </ion-item>
  <ion-item detail="true" button href="mailto:hola@adoptapet.com">
    <ion-icon name="mail-outline" slot="start" color="primary"></ion-icon>
    <ion-label>Correo Electrónico</ion-label>
  </ion-item>
</ion-list>
```

Interpretación:

En lugar de construir un sistema de mensajería interno, se delegó la responsabilidad al sistema operativo. Al inyectar el protocolo universal de WhatsApp (<https://wa.me/...>) y de correo electrónico (<mailto:...>) en el atributo href del ion-item, la vista web detecta la intención y dispara el lanzamiento nativo de la aplicación correspondiente (si está instalada), inyectando automáticamente el texto predefinido mediante parámetros URL (?text=...).

Diseño Visual y Personalización

El sistema visual se gestiona mediante CSS Custom Properties en el archivo variables.scss. Se definió una paleta de colores corporativos y se implementó un modo oscuro manual a través de un servicio global.

Tema Visual y Modo Oscuro

Se creó un servicio ThemeService que permite al usuario alternar entre tema claro y oscuro mediante un botón presente en todas las páginas.

Código de theme.service.ts:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ThemeService {
  private isDark = false;

  constructor() {
    // Detect system preference on load
    const prefersDark = window.matchMedia('(prefers-color-scheme: dark)');
    this.isDark = prefersDark.matches;
    this.applyTheme();
  }

  toggle() {
    this.isDark = !this.isDark;
    this.applyTheme();
  }

  isDarkMode() {
    return this.isDark;
  }

  private applyTheme() {
    document.documentElement.classList.toggle('ion-palette-dark', this.isDark);
  }
}
```

Interpretación:

La anotación `@Injectable({ providedIn: 'root' })` garantiza que Angular instancie el servicio como un Singleton (una única instancia compartida por toda la aplicación). El método `toggle()` accede directamente a la raíz del árbol DOM (`document.documentElement`) para inyectar o remover la clase maestra `'ion-palette-dark'`. Centralizar esta lógica en un servicio cumple con el principio de responsabilidad única (SOLID), permitiendo que cualquier componente invoque el cambio de tema sin duplicar código.

Guía de Ejecución

Requisitos previos:

- Node.js $\geq 16.x$
- npm $\geq 8.x$
- Ionic CLI: `npm install -g @ionic/cli`

Pasos para ejecutar en entorno local:



1. Clonar el repositorio desde GitHub:

```
git clone https://github.com/yhusleika/Ionic.git
```

Este comando descargará una copia local del proyecto en su máquina.

2. Navegar al directorio del proyecto:

```
cd Ionic
```

(Dentro de la carpeta clonada se encuentra el código fuente de AdoptaPet)

3. Instalar dependencias:

```
npm install
```

4. Iniciar el servidor de desarrollo:

```
ionic serve
```

(Alternativamente: npm run start)

5. La aplicación se abrirá automáticamente en <http://localhost:8100>.
6. Para simular vista móvil: abrir las herramientas de desarrollador de Chrome (F12) y activar el modo de dispositivo móvil (Device Toolbar).

Nota sobre emulación de hardware:

Durante el arranque web, es importante que el archivo main.ts invoque la función `defineCustomElements(window)` de la biblioteca `@ionic/pwa-elements`. Esto instruye al navegador a emular interfaces de hardware (como el modal de la cámara) que de otra manera solo estarían disponibles al compilar el APK o IPA.

Tecnologías Utilizadas

- Ionic Framework v7+: Componentes UI nativos (ion-menu, ion-segment, ion-card, etc.).
- Angular v17+: Enrutamiento, lazy loading, directivas estructurales, formularios reactivos.
- TypeScript: Tipado fuerte, async/await, decoradores.
- SCSS: Preprocesador CSS con variables y anidamiento.
- Capacitor: Acceso nativo a cámara y almacenamiento.

Estructura del Proyecto

La organización general del proyecto y los archivos de configuración más importantes se listan a continuación:



AdoptaPet/

```
├── src/
│   ├── app/
│   │   ├── app-routing.module.ts  (Rutas con lazy loading)
│   │   ├── app.module.ts          (Módulo raíz)
│   │   ├── app.component.html/ts/scss
│   │   ├── services/theme.service.ts
│   │   ├── inicio/                (módulo, página, estilos)
│   │   ├── informacion-personal/  (módulo, página, estilos)
│   │   └── contacto/              (módulo, página, estilos)
│   ├── theme/variables.scss        (paleta y variables CSS)
│   ├── global.scss                 (estilos globales)
│   ├── index.html
│   └── main.ts
├── angular.json
├── capacitor.config.ts
├── ionic.config.json
└── package.json
```

Servicios y Lógica Compartida

Además del ThemeService, la aplicación utiliza servicios de Angular para mantener la coherencia de los datos entre componentes. Aunque en la versión actual los datos son locales (hardcodeados en los componentes TypeScript), la estructura basada en servicios permite en el futuro conectar fácilmente con APIs REST o bases de datos en tiempo real.

Flujo de Datos

1. El navegador carga index.html → descarga el bundle principal (main.js).
2. main.ts arranca Angular mediante
platformBrowserDynamic().bootstrapModule(AppModule).
3. Angular instancia AppModule, que declara AppComponent.



4. AppComponent se renderiza en <app-root> → aparece el menú lateral.
5. El Router lee la URL actual → dirige " a '/inicio'.
6. El Router descarga el chunk de InicioPageModule (lazy loading).
7. InicioPage se renderiza dentro del ion-router-outlet.
8. Las animaciones CSS (gráfica de barras, efectos de entrada) comienzan automáticamente al estar los elementos en el DOM.
9. La navegación entre páginas descarga los chunks correspondientes solo la primera vez que se accede a ellas.



Conclusión

El desarrollo técnico de "AdoptaPet" demuestra la capacidad de orquestar soluciones de software modulares utilizando el ecosistema híbrido de Ionic y Angular. A través de la redacción manual de servicios de estado (tema oscuro), el control estricto de directivas estructurales (ngSwitch), el consumo de APIs asíncronas nativas (cámara de Capacitor) y la implementación de animaciones CSS optimizadas (gráfica con scaleY), se evidencia un completo dominio del código fuente generado. El resultado es una aplicación funcional, fluida y centrada en la adopción responsable de animales, que sirve como base sólida para futuras expansiones.



Bibliografía

- Angular. (2024). Angular Documentation: NgModules vs Standalone Components. Recuperado de <https://angular.dev/>
- Capacitor. (2024). Capacitor Camera Plugin API. Recuperado de <https://capacitorjs.com/docs/apis/camera>
- Ionic Framework. (2024). UI Components and Theming / Dark Mode. Recuperado de <https://ionicframework.com/docs/components>
- Antigraffiti (IA asistente). Utilizada como editor de código y apoyo en la generación de fragmentos y lógica de programación durante el desarrollo de AdoptaPet.
- Repositorio del proyecto: <https://github.com/yhusleika/Ionic.git>